Entrega 2 Prototipo Funcional de Aplicación Web

Cancha Central Plataforma Web de Noticias Deportivas

David Santiago Rodríguez Cruz

Politécnico Grancolombiano

John Olarte Ramos

26 de septiembre de 2026

Tabla de contenido

Introducción 3

Objetivos 3

Implementación del prototipo 4

Estructura del código 4

Maquetación de la Entrega 1 5

Correspondencia entre mockups y desarrollo 9

Funcionalidades implementadas 10

Pruebas funcionales 11

Fragmentos de código fuente 12

Anexo de código fuente 13

Repositorio, conclusiones y referencias 16

Introducción

Este documento presenta la segunda entrega de Cancha Central, una plataforma web de noticias deportivas. El propósito de esta fase fue transformar la propuesta visual de la Entrega 1 en un prototipo funcional que permitiera explorar noticias, consultar su detalle, guardar favoritos, enviar mensajes y administrar publicaciones de forma local.

La versión desarrollada utiliza HTML para la estructura semántica, CSS para el diseño responsive y JavaScript para el renderizado dinámico y las interacciones. Las noticias se obtienen desde un archivo JSON local y las acciones del usuario se conservan en localStorage. MDN Web Docs (2025a) señala que HTML organiza el contenido de una página, mientras que JavaScript permite dotar de comportamiento a la interfaz (MDN Web Docs, 2025b).

Objetivos

Objetivo general

Implementar una primera versión funcional de Cancha Central que corresponda con los mockups previamente definidos y evidencie el uso integrado de HTML, CSS y JavaScript.

Objetivos específicos

Construir las páginas Home, Listado, Detalle, Favoritos, Contacto y Administración.

Renderizar noticias mediante un archivo JSON local.

Implementar favoritos persistentes mediante localStorage.

Validar el formulario de contacto y mostrar un mensaje de confirmación.

Implementar un mini CRUD para crear y eliminar noticias locales.

Implementación del prototipo

El prototipo mantiene la identidad visual definida en la primera entrega: azul marino para la navegación y la información editorial, verde para acciones principales, amarillo como color de énfasis y marfil como fondo. El diseño responde a pantallas de computador y dispositivos móviles mediante media queries en CSS. Las tarjetas emplean fotografías deportivas almacenadas localmente en la carpeta assets, por lo que la presentación visual no depende de una conexión a internet.

Tecnologías utilizadas

Elemento	Descripción
HTML5	Estructura semántica de las seis páginas y formularios accesibles.
CSS3	Sistema visual, grid, tarjetas, diseño responsive y estados de interacción.
JavaScript	Carga dinámica del JSON, filtros, favoritos, validaciones y CRUD.
JSON local	Fuente inicial de noticias ubicada en data/noticias.json.
localStorage	Persistencia de favoritos y noticias creadas desde el panel editorial.

Estructura del código

La organización separa responsabilidades para facilitar la lectura y el mantenimiento del código. Los archivos de presentación se ubican en la raíz, los estilos en css, la lógica compartida en js, los datos en data y los recursos gráficos en assets.

```
cancha-central/
├── index.html | noticias.html | detalle.html | favoritos.html
├── contacto.html | administrar.html
├── css/styles.css
├── js/app.js
├── data/noticias.json
└── assets/futbol-*.jpg
```

Maquetación de la Entrega 1

Las siguientes pantallas corresponden a la propuesta visual utilizada como guía durante el desarrollo. La implementación funcional conserva la navegación, las tarjetas de noticias, los llamados a la acción y la identidad visual planteada en Figma.

Diseño de Figma: https://www.figma.com/design/t0vgghM0RM6XpV8sWRljBA/Cancha-Central-%E2%80%94-Mockups?node-id=0-1



Figura 1. Maquetación de la página Home de Cancha Central.

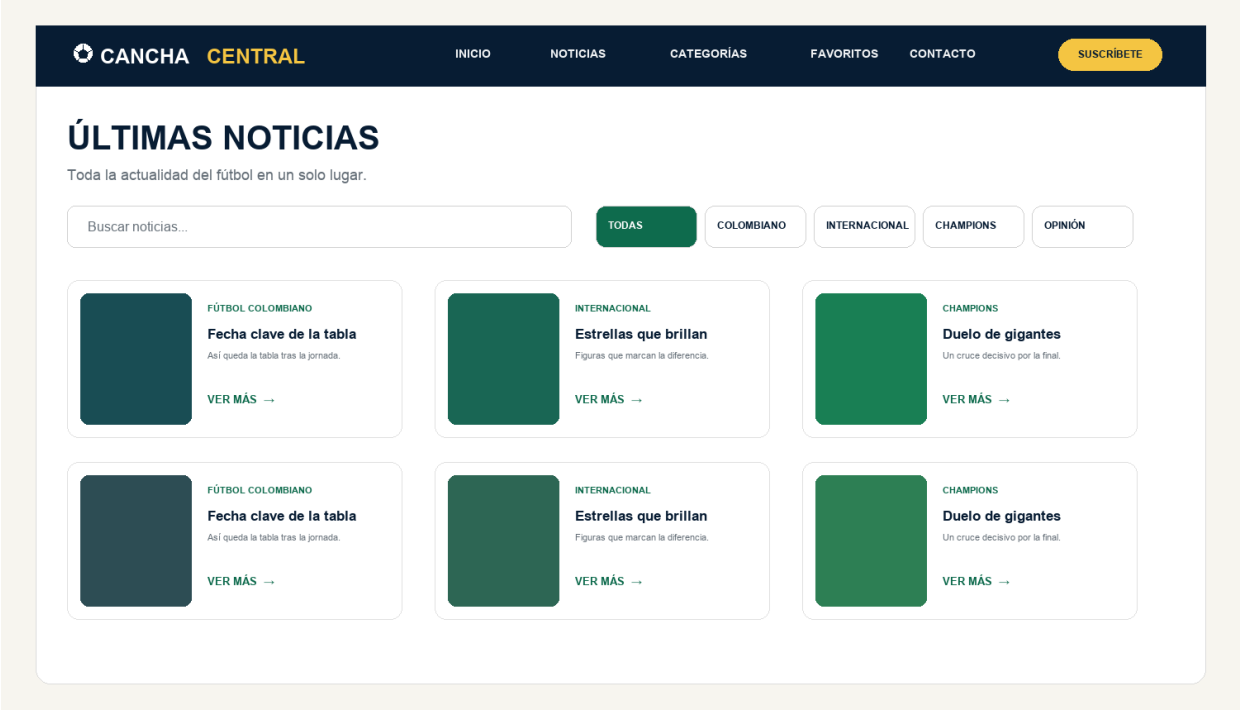


Figura 2. Maquetación del listado de noticias.

Maquetación de la Entrega 1

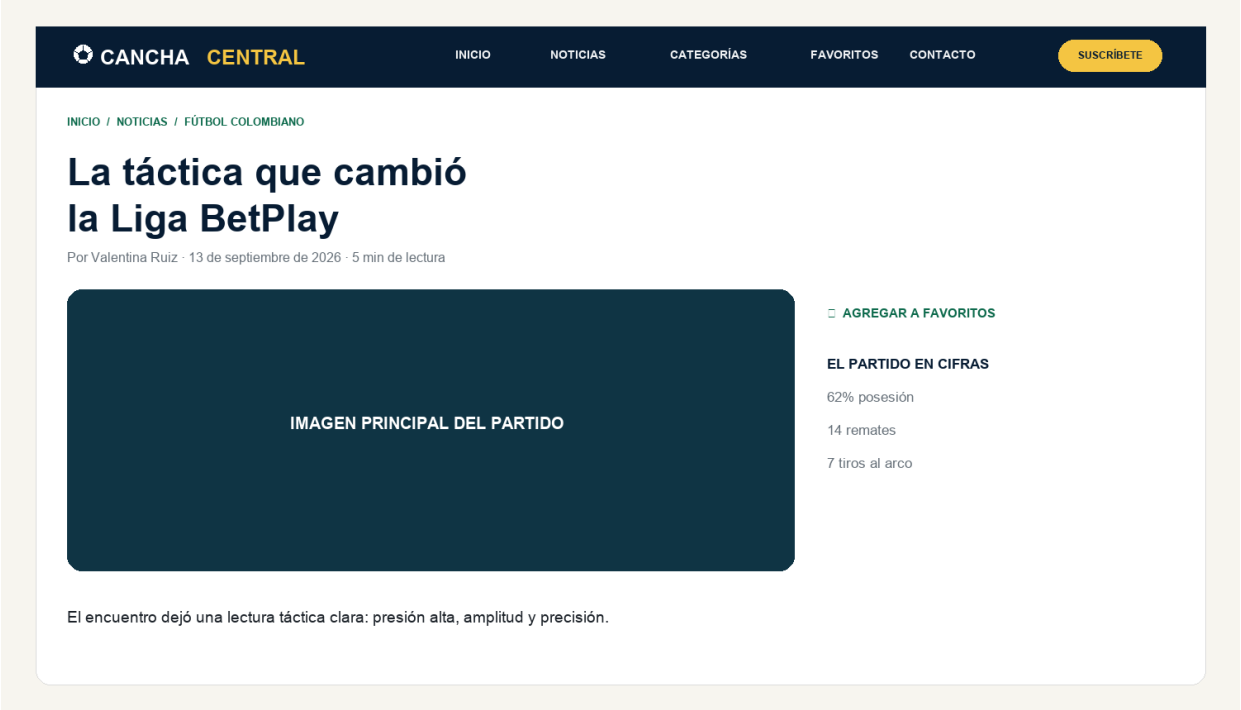


Figura 3. Maquetación del detalle de noticia.

Maquetación de la Entrega 1

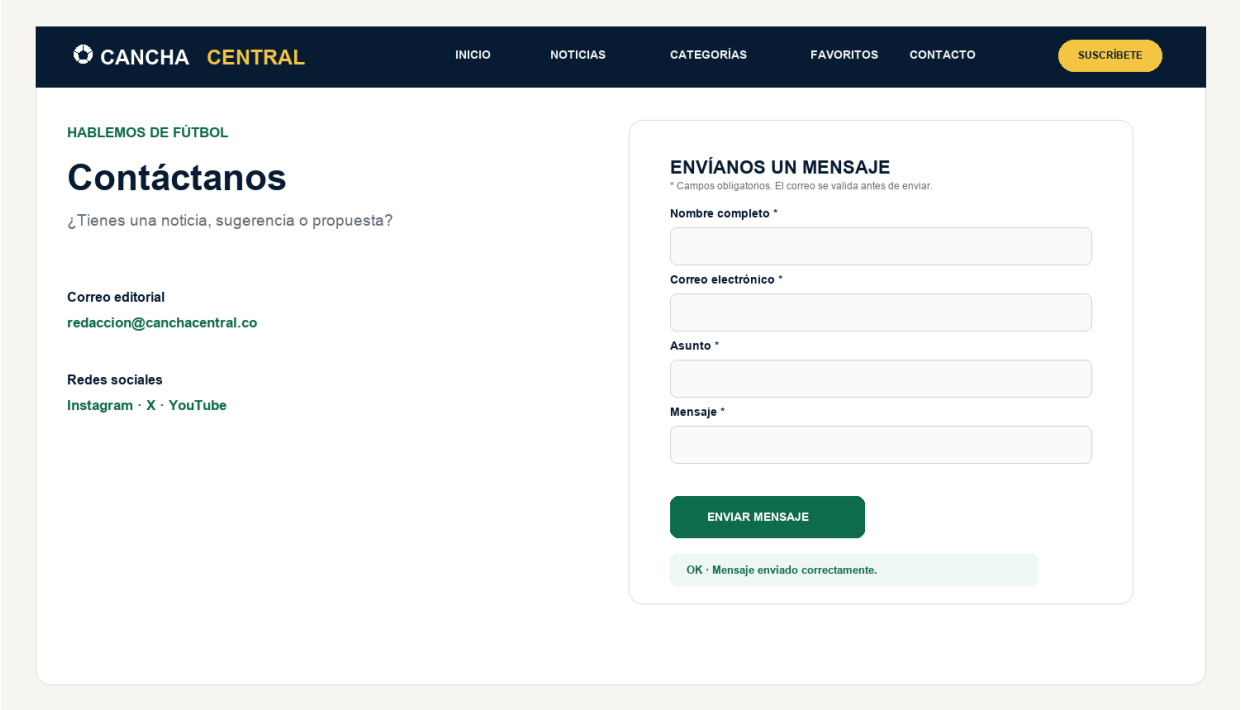


Figura 4. Maquetación de la página de contacto.

Correspondencia entre mockups y desarrollo

La nota de entrega solicita conservar la relación entre la maquetación y el prototipo funcional. La siguiente matriz evidencia cómo se trasladaron al desarrollo los elementos centrales definidos en Figma.

Elemento	Descripción
Home	Se conservan el encabezado, sección de bienvenida, llamadas a la acción, noticias destacadas y pie de página.
Listado	El catálogo mantiene tarjetas con imagen, categoría, título, descripción breve y enlace. Se añadieron búsqueda y filtros.
Detalle	La vista conserva la jerarquía de título, imagen, información editorial y acción para agregar a favoritos.
Contacto	Se conserva la composición de información editorial y formulario; se implementaron validaciones y confirmación de envío.

Las pantallas de Favoritos y Administración amplían la propuesta inicial para cubrir la interacción persistente y el mini CRUD solicitados en la Entrega 2.

Funcionalidades implementadas

Elemento	Descripción
Noticias dinámicas	noticias.html consulta data/noticias.json y genera tarjetas con imagen, categoría, título, descripción y enlace.
Búsqueda y filtros	El listado filtra las noticias por texto y por categoría sin recargar la página.
Detalle	detalle.html recupera el identificador de la noticia desde la URL y muestra su contenido completo.
Favoritos	El usuario agrega o elimina favoritos. La lista se conserva mediante localStorage, tecnología descrita por MDN Web Docs (2025c).
Contacto	El formulario valida campos obligatorios, formato de correo y longitud mínima del mensaje antes de confirmar el envío.
Mini CRUD	administrar.html permite crear noticias y eliminarlas; los cambios se almacenan localmente para la sesión posterior.

La interfaz incluye un menú de seis páginas, un pie de página uniforme y adaptación responsive. La estructura se diseñó para que el código pueda evolucionar a componentes de Angular en la siguiente entrega.

Pruebas funcionales

Se realizaron pruebas manuales en navegador usando el flujo principal de cada pantalla. Los resultados permiten comprobar el comportamiento solicitado sin requerir un servidor de base de datos.

Elemento	Descripción
Carga desde JSON	Al abrir Inicio o Noticias se generan tarjetas a partir de data/noticias.json. Resultado: correcto.
Búsqueda y filtro	Se consultaron términos y categorías; el contador y las tarjetas se actualizan sin recargar. Resultado: correcto.
Favoritos	Se agregó una noticia, se navegó a otra página y se verificó su permanencia al volver. Resultado: correcto.
Formulario	Se intentó enviar campos vacíos, correo inválido y mensaje corto; se mostraron errores. Con datos válidos aparece confirmación. Resultado: correcto.
Crear noticia	Desde Administración se registró una noticia válida y apareció en el panel y el catálogo. Resultado: correcto.
Eliminar noticia	Se eliminó una publicación creada; se retiró del panel y de favoritos, si estaba guardada. Resultado: correcto.

Estas verificaciones demuestran el uso integrado de HTML, CSS y JavaScript, así como la persistencia local requerida para esta fase del proyecto.

Fragmentos de código fuente

Carga de noticias desde JSON

```
baseNews = await fetch('data/noticias.json').then(r => r.json());
function newsCard(item) {
  return `<article class="card"> ... ${item.title} ... </article>`;
}
```

Persistencia de favoritos

```
function toggleFavorite(id) {
  const items = favorites();
  setLocal(KEYS.favorites, items.includes(id)
    ? items.filter(x => x !== id) : [...items, id]);
}
```

Validación del formulario

```
form.addEventListener('submit', e => {
  e.preventDefault();
  const bad = !field.validity.valid;
  field.classList.toggle('invalid', bad);
  if (valid) success.hidden = false;
});
```

Estos fragmentos se encuentran comentados y centralizados en js/app.js. El repositorio local contiene el código fuente completo y el archivo README con las instrucciones para ejecutar el servidor local.

Anexo de código fuente

Los siguientes extractos amplían la evidencia del desarrollo. Se presentan por responsabilidad para mostrar la organización entre estructura HTML, estilos CSS, datos JSON y lógica JavaScript.

Estructura de la página de inicio index.html

```
<body data-page="home">
  <header class="site-header"></header>
  <main id="contenido">
    <section class="hero"> ... </section>
    <div id="featured-news" class="cards cards--three"></div>
  </main>
  <script src="js/app.js"></script>
</body>
```

El atributo data-page identifica la vista actual. JavaScript inserta el encabezado y el pie de página compartidos, lo que evita repetir el mismo código en cada archivo HTML.

Datos de noticias data/noticias.json

```
{
  "id": 1,
  "title": "La táctica que cambió la Liga BetPlay",
  "category": "Fútbol colombiano",
  "image": "assets/futbol-balón.jpg",
  "description": "Análisis de la jornada..."
}
```

Anexo de código fuente

Estilos de tarjetas css/styles.css

```
.cards {
  display: grid;
  gap: 1.4rem;
}
.cards--three { grid-template-columns: repeat(3, 1fr); }
.card {
  background: #fff;
  border: 1px solid var(--line);
  border-radius: 15px;
  overflow: hidden;
  display: flex;
  flex-direction: column;
}
.card img {
  width: 100%; height: 170px;
  object-fit: cover; object-position: center top;
}
```

La cuadrícula organiza las tarjetas en tres columnas en escritorio. Las reglas responsive cambian el número de columnas en pantallas pequeñas para conservar la legibilidad.

Carga de datos y generación de tarjetas js/app.js

```
async function init() {
  renderLayout();
  try {
    baseNews = await fetch("data/noticias.json")
      .then(response => response.json());
  } catch {
    // Se informa al usuario si el JSON no está disponible.
    return;
  }
  const page = document.body.dataset.page;
  const initPage = { home: initHome, news: initNews,
    detail: initDetail, favorites: initFavorites,
    contact: initContact, admin: initAdmin }[page];
  if (initPage) initPage();
}

function newsCard(item) {
  return `<article class="card"> ... ${item.title} ... </article>`;
}
```

Anexo de código fuente

Favoritos y almacenamiento local js/app.js

```
function favorites() {
  return getLocal(KEYS.favorites, []);
}

function toggleFavorite(id) {
  const items = favorites();
  const updated = items.includes(id)
    ? items.filter(item => item !== id)
    : [...items, id];
  setLocal(KEYS.favorites, updated);
}
```

La función alterna el identificador de una noticia en localStorage. De esta manera, los favoritos se conservan cuando el usuario cambia de página o vuelve a abrir el sitio.

Validación del formulario y creación de noticias

```
form.addEventListener("submit", event => {
  event.preventDefault();
  const invalidEmail = !form.elements.email.validity.valid;
  const shortMessage = form.elements.message.value.trim().length < 10;
  if (invalidEmail || shortMessage) return;
  form.reset();
  success.hidden = false;
});

// El panel Administración crea una noticia y la guarda en localStorage.
setLocal(KEYS.news, [item, ...getLocal(KEYS.news, [])]);
```

Estos bloques corresponden al código fuente completo incluido en el repositorio. Los comentarios explican las decisiones de carga, persistencia, validación y creación de noticias.

Repositorio del proyecto

El código fuente se encuentra publicado en el repositorio público de GitHub Cancha Central. URL del repositorio: https://github.com/SantiRCruz/cancha-central. El proyecto incluye un archivo README con instrucciones de ejecución mediante un servidor local.

Conclusiones

La Entrega 2 demuestra la transición de la maquetación hacia un prototipo web funcional. El sitio conserva la identidad de Cancha Central y resuelve los recorridos principales de un usuario: descubrir noticias, filtrarlas, consultar su detalle, guardarlas como favoritas y contactar al equipo editorial.

El uso de un archivo JSON y localStorage permite evidenciar una gestión básica de datos sin requerir un servidor. Esta arquitectura facilita la evolución futura hacia Angular, componentes reutilizables y un despliegue público en la entrega final.

Referencias

MDN Web Docs. (2025a). HTML: HyperText Markup Language. Mozilla. https://developer.mozilla.org/en-US/docs/Web/HTML

MDN Web Docs. (2025b). JavaScript. Mozilla. https://developer.mozilla.org/en-US/docs/Web/JavaScript

MDN Web Docs. (2025c). Window: localStorage property. Mozilla. https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage

Figma. (s. f.). Figma. https://www.figma.com/